The data we are building on

Findings and proposed agreements · Sankalp · Civic Operations · Group 23 · 5 October 2026

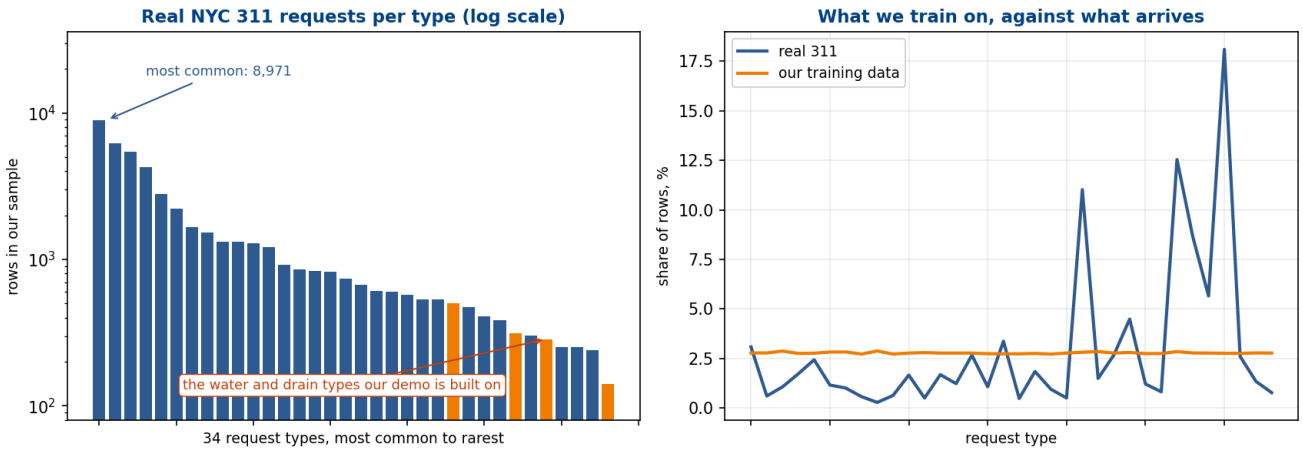
Why I wrote this. We each own our own components, which is right, but we all draw on one dataset. I went through it properly and found things that will quietly spoil our results if we do not handle them: the data is heavily imbalanced, the part our demo is built on is the thinnest part of all, and our training text does not look like what arrives.

Nothing here tells anyone how to build their component. It is findings, then a short list of things worth agreeing so four separate pieces still add up to one system.

1. What we actually hold

Table	Rows	Real or synthetic	Who leans on it
requests_real	49,558	Real NYC 311, 5 boroughs, 34 types	Everyone
daily_demand_weather	~8,600 borough-days	Real counts + Open-Meteo	Predictive alerting
synthetic_nlp_utterances	18,654	Synthetic, template-generated	Intake bot, emergency
nlp_gold_test	152	Hand-written by us	The only honest NLP test
synthetic_citizens / reporter_links	5,000 / ~25k	Synthetic	Credibility, fake detection
synthetic_crews, queue snapshots, urgency scenarios	136 / 100 / 1,000	Synthetic	Dispatch, prioritisation, urgency

2. Finding 1: the data is 64 to 1 imbalanced



Left: every request type in our sample, log scale. Right: our training distribution against the real one.

The most common type, **Illegal parking or blocked hydrant**, has **8,971 rows**. The rarest, **Street flooding**, has **141**. That is a ratio of about **64 to 1**. Eight of the 34 types have fewer than 350 rows.

Accuracy will look respectable while the rare types fail completely, because the common types carry the average. Any model any of us trains on this will quietly learn to predict noise and parking.

3. Finding 2: the story we tell is the thinnest data we own

Complaint family	Rows	Share of the sample
NOISE_PARKING	23,570	47.6%
HOUSING	10,357	20.9%
SANITATION	5,212	10.5%
STREETS	5,002	10.1%
WATER_SEWER	2,317	4.7%
PARKS_TREES	1,639	3.3%
BUILDINGS	1,054	2.1%
HEALTH	407	0.8%

Our entire demonstration, the proposal and the presentation are built on a flooded drain near a school. **Water and sewer is 4.7% of our rows.** Street flooding itself has **141 rows** and clogged catch basins **286**, against 8,971 for illegal parking. If we want to defend the flooding example in October, we have to deal with this, either by oversampling those types when we sample again, or by being honest that the example is illustrative and our evidence rests on the common types.

4. Finding 3: our borough sampling is not the city

Borough	Rows in our sample	Real share of 311 (last 12 months)
Brooklyn	9,980	1.21M, about 31%
Queens	9,975	963k, about 24%
Bronx	9,983	850k, about 22%
Manhattan	9,907	767k, about 19%
Staten Island	9,713	160k, about 4%

We deliberately took about 10,000 per borough so every borough had enough rows to model. That was a reasonable choice, but it has two consequences we must not forget. Any percentage we quote from this sample is **not a citywide percentage**, and any model that uses borough as a feature has learned a borough balance that does not exist. Staten Island is over-represented by roughly a factor of six.

5. Finding 4: a quarter of our outcome labels cannot be used as truth

Outcome from the closing notes	Rows	Share
UNFOUNDED	18,306	36.9%
GENUINE	9,718	19.6%
SUBSIDED	7,788	15.7%
ADMINISTRATIVE	5,740	11.6%
NO_ACCESS	3,479	7.0%
DUPLICATE	2,912	5.9%
REFERRED	1,164	2.3%
NO_NOTE	451	0.9%

Grouped for training, that is **17,506 genuine**, **18,306 unfounded**, and **13,746 inconclusive**, which is 28% we must exclude rather than guess at. Separately, only **81%** of rows have a resolution time we can trust, and **2.3%** have coordinates outside New York.

The useful half of this finding: unfounded outcomes are not evenly spread. They cluster hard by request type, which is the single strongest signal available on day one for judging whether a report will turn out to be real.

6. Finding 5: we train on a flat world and test on a steep one

Our generated training text is close to uniform, about 442 examples per intent. Real life is 64 to 1. Two things follow. The model has no idea what is common, and two classes are starved: **OTHER has 6 training rows** and SMALLTALK has 9.

> That is not a theory. It is the measured cause of the bug we already found: “my uber driver was rude” and “can I get a copy of my birth certificate” are confidently routed to a sanitation crew, because the model was shown six examples of what out-of-scope looks like.

The fix is built and ready to test, not yet proven. The next run trains the same models on a distribution that matches real 311 frequencies, with and without realistic text noise, and scores all of them on the gold set. At smoke-test scale the four variants are indistinguishable, with confidence intervals that overlap almost completely, which is itself evidence for the point in section 7: with 152 test lines we cannot tell 0.85 from 0.90.

7. Finding 6: our test set was thin exactly where the city is busiest, and is now fixed

The original test set was 152 hand-written lines across 37 intents. Seven intents had **two** lines. Illegal parking, which is 18% of all real requests, had **three**. A macro average over classes with two examples is extremely noisy, and that, rather than genuine similarity, is why our three models looked tied in the last run.

This is done. No one needs to write anything. 156 further lines were written on 5 October, bringing the set to **308 lines with at least eight per intent**. Each one is grounded in a descriptor sub-type that really occurs under that request type, and written in mixed registers: typos, missing punctuation, run-on voice transcripts, terse messages, shouting, and non-native phrasing. They are checked automatically against the original set, the training data and the out-of-scope lines so nothing is reused.

They are kept as a separate slice, and that already told us something uncomfortable. The new lines deliberately avoid the obvious keyword for each type. Scored against them:

Model	Original 152 lines	New 156 lines
Keyword rules	0.720	**0.359**
TF-IDF + SVM	0.872	**0.539**
MiniLM embeddings	0.888	**0.657**

> Change the wording and performance roughly halves. **Our models have been learning the vocabulary, not the meaning.** That is the same lesson as validation against the gold set, one level deeper. The new slice is harder on purpose, so it is a lower bound rather than the truth, and neither slice should ever be quoted on its own.

8. Finding 7: there is no real citizen text, and that is worth saying out loud

I checked whether we could train on real 311 text instead of generated sentences. We cannot. The entire dataset contains only **253 distinct descriptor strings**, such as “Illegal Parking / Blocked Hydrant”. They are dropdown categories chosen by an agent, not words a citizen wrote. A model trained on them would score almost perfectly and would have learned nothing about language.

So the synthetic utterances and the hand-written gold set are not shortcuts, they are the only honest option available from open data. If an examiner asks why we did not use real text, that is the answer, and it is a good one.

9. Fixed scenarios, so the system can actually be tested

If a citizen can type anything and the location changes every time, nothing downstream can be tested. The agency moves, the crew skill moves, the urgency moves, and the route moves, so there is no stable expected answer to compare against. The portal therefore offers a **fixed menu of reporting scenarios**, each pinned to a **fixed address**.

`data/reference/demo_scenarios.csv` holds 34 of them, one per request type, and they are drawn from the real data rather than invented: the complaint type and descriptor are the ones NYC actually records, and the address and coordinates are taken from a real row of that type. Each scenario carries the answer every component should produce.

Column	What it is for
scenario_id, request_type_code, citizen_label	Which case this is
scenario_text	What the citizen sends, so intake always receives the same words
complaint_type, descriptor	The official NYC wording, so our vocabulary matches 311's
borough, incident_address, latitude, longitude	A real, fixed location, so routing and distance are stable
expected_agency, expected_skill, expected_urgency	The answer to assert against
est_service_minutes	So dispatch has a duration to plan with
rows_in_real_311	How common this type actually is, so we know which cases matter most

This gives every one of us the same thirty-four cases to test against, end to end, and it is what makes an integration test possible later: the same scenario goes in, and each owner can check that their component produced what the table says it should.

One rule attached to it. *These scenario lines are not in either test set and must never be added to one. Demonstrating the system on the cases it was tuned against is the oldest mistake in the book.*

10. What each of us should watch for

Component	Where the data will bite	Cheapest defence
Intake bot	Flat training prior; OTHER starved	Realistic prior, out-of-scope lines, abstain threshold
Emergency safeguard	19 positive examples in gold	Report recall with an interval; never tune on gold
Agency routing	64:1 types; rules cover the head, the tail is thin	Rules first, model for the tail, report per-class recall
Duplicates	Our sample keeps same-incident reports together, which flatters pair precision	Report false merge rate separately
Fake and unfounded	28% inconclusive labels; type base rate dominates	Exclude inconclusive; always show a type-only baseline
Credibility	No real citizen identities exist	State the synthetic evaluation openly
Resolution time	Only 81% of rows usable; heavy tail	Predict a range, report interval coverage
Urgency and priority	No ground-truth urgency label at all	Independent rubric, measure agreement, not accuracy
Dispatch	Crews are synthetic	Compare against baselines, not against reality
Predictive alerting	Equal-borough sampling distorts volumes	Model per borough, never pool across

		them

11. Which columns are safe to use, and when

This is the practical version of the shared-table idea. The 54 columns are not equally usable: some are known the moment a citizen reports, some only exist once a crew has closed the case. If the two sit side by side and nobody marks which is which, somebody will train on a closure column by accident and report an accuracy of 0.98 that means nothing. The fix costs nothing: know which group a column is in before you use it.

When it becomes known	Columns	Safe to use as a feature?
At intake, the moment the request is filed	created_at, hour, dow, month, is_weekend, is_holiday, borough, incident_address, incident_zip, bbl, latitude, longitude, geo_valid, location_type, address_type, complaint_type, descriptor, request_type_code, complaint_family, channel	Yes, for every component
At intake, from our own catalogue	catalog_agency, required_skill, default_urgency, est_service_minutes, request_type_label	Yes. These are lookups, not outcomes
At intake, from the weather service	wx_precip_mm, wx_rain_mm, wx_snow_cm, wx_tmax_c, wx_tmin_c, wx_gust_kmh	Yes, but only the forecast for prediction, never the observed value for the same day
Assigned by the city, so a target rather than a feature	agency, agency_name, agency_name_std	This is what agency routing predicts. Never feed it into the model that predicts it
Only at closure, hours or weeks later	closed_date, status, resolution_description, resolution_action_updated_date, resolution_hours, log_resolution_hours, resolution_valid, outcome_class, verified_outcome	**No. These are labels.** Using any of them as a feature is leakage
Bookkeeping	unique_key, request_id, source, is_synthetic, split, sample_window_id	Keys and housekeeping only, never a feature

> **The two that will catch somebody out.** `resolution_description` is the closing note, and it is exactly what we derive the genuine-or-unfounded label from, so a fake-detection model that reads it will look perfect and be worthless. And `agency` is the answer the routing component is supposed to produce.

When we come to joining our components together, this is also the list we will need: each of us says which columns we consume and which we produce, and anything consumed has to be available at the stage where the component runs. That conversation is worth having once everybody knows their own inputs and outputs, not before.

12. What I propose we agree

Short, and only the things that stop four separate components contradicting each other later.

1. **Split by time, never at random.** The same incident appears several times; a random split puts the same event on both sides and every score comes out flattering. The shared table already has a time-based split column.
2. **Nobody trains on the gold set.** It is our only unseen test. If anyone needs more examples, we write more.
3. **The 34-type catalogue stays fixed.** If someone needs a change, say so in the group chat, because it changes the label set for the bot, routing, urgency and everything downstream.
4. **Quote n with every metric**, and prefer macro with a per-class table over a single headline number, so a rare type failing cannot hide behind a common one.

5. **Inconclusive outcomes stay excluded from truth labels.** Guessing them would teach every model the same error.
6. **Say what you dropped.** One line in your notebook: how many rows, and why. Most of our disagreements later will be two people quoting different numbers from differently filtered data.

What stays entirely yours. How you resample, weight or augment inside your own component, which algorithms you compare, how many, and how you tune them. The agreements above are about the shared table and the shared test, not about your method. If a rebalanced training set works for you, use it and report it.

13. What I suggest we do next

Action	Who	Status
Fill the thin intents in the test set, 156 new lines	Done, 5 October	308 lines, 8 per intent
Fixed scenario catalogue with real addresses	Done, 5 October	34 scenarios
Remove the two test lines that had leaked into training	Done	In the notebook
Correct the type distribution for how we sampled boroughs	Done	9 points of shift
Run the version 2 NLP benchmark on a GPU	Mohammad, or us overnight	Notebook verified, ready
Decide whether to re-sample 311 with proportional boroughs	Team decision	Open
Each owner notes rows used, filters and split in their notebook	Each owner	15 minutes each

None of this is urgent in the sense of today, but all of it is cheaper now than in the week before submission, when a reviewer asks why the rare classes were never evaluated.

Appendix A. Every term in this document, in plain English

Nothing here is complicated once the name is unpacked. If a word in this document was unfamiliar, it is explained below.

About the data

Term	What it means
Gold set	A set of test examples where a human decided the correct answer, so it is the closest thing we have to truth. Ours is 308 messy, hand-written citizen messages. It is called gold because it is the standard everything else is measured against.
Training set	The examples a model learns from. Ours are generated from templates.
Validation set	Examples held back during training, used to tune settings. Ours come from the same generator as the training data, which is why they flatter the models.
Test set	Examples the model has never seen in any form, used once, at the end. The gold set is ours.
Leakage	When something from the answer sneaks into the inputs, so the model looks brilliant and is useless. In our data the closing note is the answer for fake detection, so using it as a feature is leakage.
Class imbalance	Some categories appear far more often than others. Ours runs 88 to 1 between illegal parking and street flooding.
Class prior	How often each category is expected to appear. Our training data has a flat prior, every type equally likely. The real world does not.
Base rate	How common something is before you look at any evidence. Noise complaints have a much higher base rate of turning out to be unfounded than water main breaks do.
Post-stratification	Correcting a sample that was not drawn proportionally. We took 10,000 rows per borough, so we reweight by each borough's true share of the city before using the frequencies.
Augmentation	Making extra training examples by corrupting real ones: typos, lost punctuation, shouting, truncation. It teaches a model that messy text means the same thing.
Out-of-scope	A message that is a genuine request but not something a crew can be sent to, such as asking how to renew a licence.

About the measurements

Term	What it means
Accuracy	The share of answers that were right. It flatters you when one category dominates: always answering the most common class can score 70% and be worthless.
Precision	Of the things we flagged, how many really were that thing. Low precision means false alarms.
Recall	Of the things that really were that thing, how many we caught. Low recall means misses. For emergencies, recall is the number that matters.
F1	Precision and recall combined into one number, in a way that punishes being lopsided.
Macro-F1	F1 worked out separately for each of the 37 categories and then averaged, so a rare category counts as much as a common one. It is the headline number for our intent model.
Weighted F1	The same, but common categories count for more. Higher, and less honest about rare ones.
Span F1	For extracted text rather than labels. A predicted address counts only if it matches the human-marked span exactly, start and end.
Confidence interval	The range the true value is probably in. With 152 test lines ours were so wide that models two points apart could not be told apart.
Bootstrap	How we work that range out: re-score the same test set hundreds of times on random resamples of itself, and see how much the number moves.
Cohen's kappa	Agreement with a human, after subtracting the agreement you would get by guessing. Weighted kappa also counts how badly wrong a mistake was.
Calibration	Whether a stated probability is honest. If a model says 80%, it should be right about 80% of the time. Brier score and expected calibration error measure this.
MAE and RMSE	The average size of the error when predicting a number rather than a category.

Poisson deviance	The right way to measure error when what you are predicting is a count of things.

About the models

Term	What it means
Intent classification	Working out which of the 34 request types a message is about.
Slot filling	Pulling the structured details out of free text: address, cross street, landmark, time.
BIO tags	How slot answers are marked word by word: Beginning of a span, Inside it, or Outside it.
TF-IDF	Counting words, weighted so that common words count for less. Fast, tiny, and purely lexical: it has no idea that drain and catch basin are related.
n-gram	A run of n words or characters. Character n-grams are what let a model survive typos.
Embedding	A sentence turned into a list of numbers, where similar meanings sit close together. This is how a model can tell that two differently worded reports are the same problem.
Transformer	The model architecture behind modern language models. DistilBERT, RoBERTa and DeBERTa are three of them, at 66, 125 and 142 million parameters.
Fine-tuning	Taking a model that already knows English and training it further on our task.
Epoch	One full pass through the training data. Too many, and the model memorises it.
Learning rate	How big a step the model takes when it corrects itself. The single most important setting.
Batch size	How many examples it looks at before each correction. Bigger needs more memory.
Seed	The starting point for the random numbers. Fixing it makes a run repeatable; changing it shows how much of the result was luck.
Hyperparameter search	Trying many combinations of those settings to find a good one. Ours tries 18 random combinations rather than every possible one.
Class weights and focal loss	Two ways to stop a model ignoring rare categories: make their mistakes cost more, or make the easy, common cases count for less.
Abstain threshold	The confidence below which the bot should say it is unsure and pass the message to a person, instead of guessing.

About the methods used elsewhere in the project

Term	What it means
Beta distribution	How we track whether a citizen has been right before: two running counts, good reports and bad ones, starting at two each so nobody begins at zero or at one hundred.
Decay	Letting old evidence count for less as time passes, so a damaged score can be recovered.
Poisson regression	The right model for predicting counts. It predicts the logarithm of the count, so the factors multiply: 38 mm of rain does not add reports, it multiplies the day.
Gradient boosting	Many small decision trees in sequence, each fixing the errors of the ones before. Good at catching combinations of conditions.
DBSCAN and HDBSCAN	Ways of grouping points that sit close together, used to spot several reports of one incident. They find the groups themselves rather than being told how many to look for.
A-star search	A route-finding method that uses an estimate of the remaining distance to avoid exploring the whole map. Same answer as Dijkstra, far less work.
Genetic algorithm	Searching for a good crew schedule by keeping the better attempts, combining them and mutating them, over many generations.
Ablation	Switching one component off to see how much worse things get, which is how we will show what each one is worth.
Replay	Running real days of history through the system as if they were happening now, to compare it against what actually happened.